A Comparative Study of SAT Encodings and Exact Solvers for the N-Queens Problem

Nguyen Nhat Minh[a,*]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, Vietnam National University, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens
Boolean satisfiability
cardinality encoding
constraint programming
mixed integer programming
exact algorithms

ABSTRACT

The N-Queens problem asks for a placement of N queens on an $N \times N$ chessboard such that no two queens share a row, column, or diagonal. This study implements a SAT-based solver and compares pairwise, sequential-counter, and bitwise at-most-one encodings, then evaluates a provisional SAT baseline against CP-SAT and mixed-integer programming solvers. Repeated local runs show that Pairwise has the lowest median SAT runtime at three of the four repeatedly measured sizes from $N = 8$ to $N = 64$; Sequential Counter is fastest at $N = 16$. A matched three-run cross-solver batch covers ten board sizes up to $N = 128$. All five methods solve every tested case, and all 150 results pass an independent validator. In this batch, SAT has the lowest median at $N = 8, 16$, Gurobi at $N = 24, 31, 45, 64$, CPLEX MIP at $N = 32$, and CPLEX CP at $N = 44, 100, 128$. The results are a repeated local comparison, with conclusions limited by three runs per configuration.

1. Introduction

The N-Queens problem is a classical constraint satisfaction problem in which N queens must be placed on an $N \times N$ board so that no two queens attack one another. Although the problem has a simple statement, it is a useful benchmark for comparing exact solving paradigms because the number of possible placements grows rapidly with N and the constraints can be represented using several modelling formalisms.

This study focuses on a SAT formulation and compares it with exact methods based on constraint programming and mixed integer programming. The SAT part is implemented with PySAT and the Glucose3 solver [3]. Three at-most-one (AMO) encodings are evaluated while the underlying SAT solver and N-Queens constraints remain fixed. This design isolates the effect of the cardinality encoding [5].

The contributions of this work are:

- a reusable SAT encoding architecture for N-Queens;
- an implementation of pairwise, sequential-counter, and bitwise AMO encodings;
- implementations of CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi MIP models;
- a common validator and benchmark result schema;
- a repeated comparison of model size, execution time, solver status, and runtime performance.

2. Problem Definition and Mathematical Models

Let $x_{r,c}$ denote whether a queen is placed at row r and column c, where $0 \le r, c < N$. The N-Queens constraints require exactly one queen in each row and column and at most one queen on each diagonal.

For SAT and MIP, $x_{r,c}$ is a Boolean or binary decision variable. The row constraints are:

$$\sum_{c=0}^{N-1} x_{r,c} = 1 \qquad \forall r. \tag{1}$$

*Corresponding author.
23021631@vnu.edu.vn (N.N. Minh)
ORCID(s):

The column constraints are:

$$\sum_{r=0}^{N-1} x_{r,c} = 1 \qquad \forall c. \tag{2}$$

For each main diagonal and anti-diagonal, the corresponding variables satisfy an at-most-one constraint. Main diagonals are grouped by $r - c$ and anti-diagonals by $r + c$.

The CP and CP-SAT models use N integer variables q_r, where q_r is the column assigned to row r. They enforce:

$$\text{AllDifferent}(q_r), \quad \text{AllDifferent}(q_r + r), \quad \text{AllDifferent}(q_r - r). \tag{3}$$

3. SAT-Based Approach

3.1. CNF Encoding

Each board position is mapped to a SAT variable using:

$$v(r, c) = rN + c + 1. \tag{4}$$

Exactly-one constraints are decomposed into an at-least-one clause and an AMO constraint. The implementation uses a shared PySAT `IDPool` whose auxiliary variables start after the N^2 original board variables. This prevents auxiliary-variable collisions across rows, columns, and diagonals.

3.2. AMO Encodings

The pairwise encoding adds one binary clause $\neg x_i \vee \neg x_j$ for every pair of literals. It introduces no auxiliary variables but can produce a quadratic number of clauses.

The sequential-counter encoding introduces auxiliary variables representing partial prefix counts. It generally provides a more compact clause structure than pairwise encoding for larger cardinality constraints, at the cost of additional variables.

The bitwise encoding introduces binary selector variables. It provides another trade-off between the number of auxiliary variables and clauses. All three encodings are supplied with the same N-Queens constraints and solved by Glucose3.

3.3. Solution Decoding and Validation

If the SAT instance is satisfiable, the model is decoded into the representation $q_r = c$. The decoded placement is then checked independently. The validator verifies the board size, column bounds, column uniqueness, main-diagonal uniqueness, and anti-diagonal uniqueness. This validation is separate from the solver model implementation.

4. Compared Exact Solvers

The study compares the SAT solver with four exact alternatives. OR-Tools CP-SAT [4] uses the integer-variable formulation and records conflicts and branches where available. CPLEX CP Optimizer [2] uses the same high-level all-different formulation. CPLEX MIP and Gurobi MIP [1] use binary board-position variables and linear row, column, and diagonal constraints.

The benchmark runner normalizes all solver outputs into a common schema containing status, validity, model-size fields where available, build or encoding time, solve time, total time, raw status, and error information.

5. Experimental Setup

5.1. Hardware and Software

The benchmarks were run on a Dell Inspiron 15 3511 with an 11th-generation Intel Core i5-1135G7 processor (4 cores, 8 logical processors) and 8 GB RAM, using Windows 11 Pro (build 22631) and Python 3.11.8. The installed software versions were PySAT 1.9.dev16, OR-Tools 9.15.6755, Gurobi 13.0.3, DOcplex 2.32.264, and CPLEX 22.2.0. CPLEX MIP and CP Optimizer passed separate 1,100-variable full-version license checks. Gurobi passed a 2,100-variable academic-license check. The full benchmark set reported here was rerun in the current environment. The repository's last recorded full test-suite run passed 43 tests; pytest was not rerun as part of this benchmark refresh.

5.2. Benchmark Protocol

The repeated SAT encoding experiment covers $N = 8, 16, 32, 64$, with five runs for $N = 8, 16, 32$ and three runs for $N = 64$. Exploratory runs at $N = 96, 128$ are excluded because earlier attempts reached the 600-second timeout. The results therefore support comparisons only over the repeated range and do not establish statistically stable performance.

The matched cross-solver batch uses Pairwise SAT at $N = 8, 16, 24, 31, 32, 44, 45, 64, 100, 128$, with three runs per method and size. The $N = 31/32$ and $N = 44/45$ pairs retain the earlier CPLEX and Gurobi license-boundary cases; all solvers now solve these sizes with the active licenses. The complete five-method batch was rerun in one session after Gurobi academic-license activation. $N = 8, 16, 24$ provide small and medium reference cases; $N = 64, 100, 128$ extend the comparison to larger instances. A 600-second per-configuration timeout is retained. Timeouts and license rejections are stored as explicit statuses, not treated as runtimes. The tables report medians of three total times; the raw CSV preserves all 150 outcomes.

Pairwise was selected as the provisional SAT baseline for the cross-solver comparison. In the latest repeated encoding batch, it had the lowest median total time at $N = 8, 32, 64$, while Sequential Counter was fastest at $N = 16$ (0.0093 s versus 0.0169 s for Pairwise). At $N = 8$, Pairwise was fastest at 0.0013 s, with Sequential Counter at 0.0015 s and Bitwise at 0.0020 s. Pairwise is a practical choice for the larger repeatedly measured cases in this dataset, but it does not dominate every size. This choice is limited to the measured PySAT/Glucose3 implementation and does not establish that Pairwise is best at every size or with every SAT solver. Sequential Counter and Bitwise remain in the encoding comparison; exploratory sizes $N = 96, 128$ are excluded because no complete repeated batch is available.

5.3. Reproducibility

Source code, benchmark outputs, summary tables, and figures are maintained in the project repository:

`https://github.com/nNm205/n-queens-solving`

6. Experimental Results

6.1. SAT Encoding Comparison

Table 1 reports the deterministic size of each complete SAT formula. Variable counts include the N^2 board variables and any encoding auxiliaries; clause counts include the full N-Queens formula. The at-least-one clauses are common to all three encodings, so the size differences arise from their AMO constraints. Pairwise uses the fewest variables, while Sequential Counter produces the fewest clauses at every reported size. Bitwise lies between them on both measures at all four sizes.

Table 1
SAT formula size by encoding. Counts are deterministic for each N.

Encoding	N	Variables	Clauses
Pairwise	8	64	744
Sequential Counter	8	270	600
Bitwise	8	170	724
Pairwise	16	256	6,352
Sequential Counter	16	1,182	2,728
Bitwise	16	568	3,924
Pairwise	32	1,024	52,640
Sequential Counter	32	4,926	11,592
Bitwise	32	1,846	19,796
Pairwise	64	4,096	428,864
Sequential Counter	64	20,094	47,752
Bitwise	64	6,132	95,572

Table 2 reports median total times from the latest repeated local batch. Pairwise is fastest at $N = 8, 32, 64$, while Sequential Counter leads at $N = 16$ (0.0093 s versus 0.0169 s). Bitwise is slower at $N = 16, 32, 64$. The formula-size trade-off is clear: Pairwise uses fewer variables but more clauses, whereas Sequential Counter reduces clauses with additional auxiliary variables. Neither count alone explains runtime. These medians describe one machine and should not be read as universal rankings.

Table 2
Median total time in seconds for the repeated SAT encoding batch.

N	Pairwise	Sequential counter	Bitwise	Repeats
8	0.0013	0.0015	0.0020	5
16	0.0169	0.0093	0.1088	5
32	0.0891	0.5117	16.7465	5
64	0.6686	12.3846	22.6755	3

Exploratory attempts at $N = 96, 128$ reached the 600-second per-configuration timeout and are excluded from the reported encoding comparison. Pairwise is used provisionally for the cross-solver comparison because it is fastest at three of the four repeated sizes and has substantially lower medians than Sequential Counter at $N = 32$ and $N = 64$. Sequential Counter is faster at $N = 16$, so the measured data do not support a claim that one encoding wins at every size. The clause and variable counts are reported as separate model-size trade-offs.

6.2. Cross-Solver Comparison

Table 3 reports medians from the matched three-run Pairwise comparison. It includes all five methods: SAT, CP-SAT, CPLEX CP, CPLEX MIP, and Gurobi MIP. All 150 solutions passed the independent validator.

Table 3
Median total time (seconds) from three Pairwise cross-solver runs per configuration.

N	SAT	CP-SAT	CPLEX CP	CPLEX MIP	Gurobi MIP
8	0.003	0.034	0.059	0.039	0.010
16	0.037	0.074	0.057	0.046	0.046
24	0.172	0.119	0.062	0.059	0.016
31	0.197	0.209	0.090	0.088	0.084
32	0.238	0.215	0.089	0.073	0.079
44	0.528	0.427	0.064	0.122	0.150
45	0.402	0.217	0.047	0.054	0.016
64	0.727	0.389	0.036	0.085	0.036
100	2.765	0.979	0.044	0.171	0.059
128	6.183	1.935	0.049	0.312	0.146

The latest matched results show that all five methods solve all ten tested sizes. SAT has the lowest median at $N = 8, 16$; Gurobi at $N = 24, 31, 45, 64$; CPLEX MIP at $N = 32$; and CPLEX CP at $N = 44, 100, 128$. At $N = 16$ and $N = 64$, the winning medians are close to the next method. These rankings are based on three runs on one machine and do not establish general solver performance.

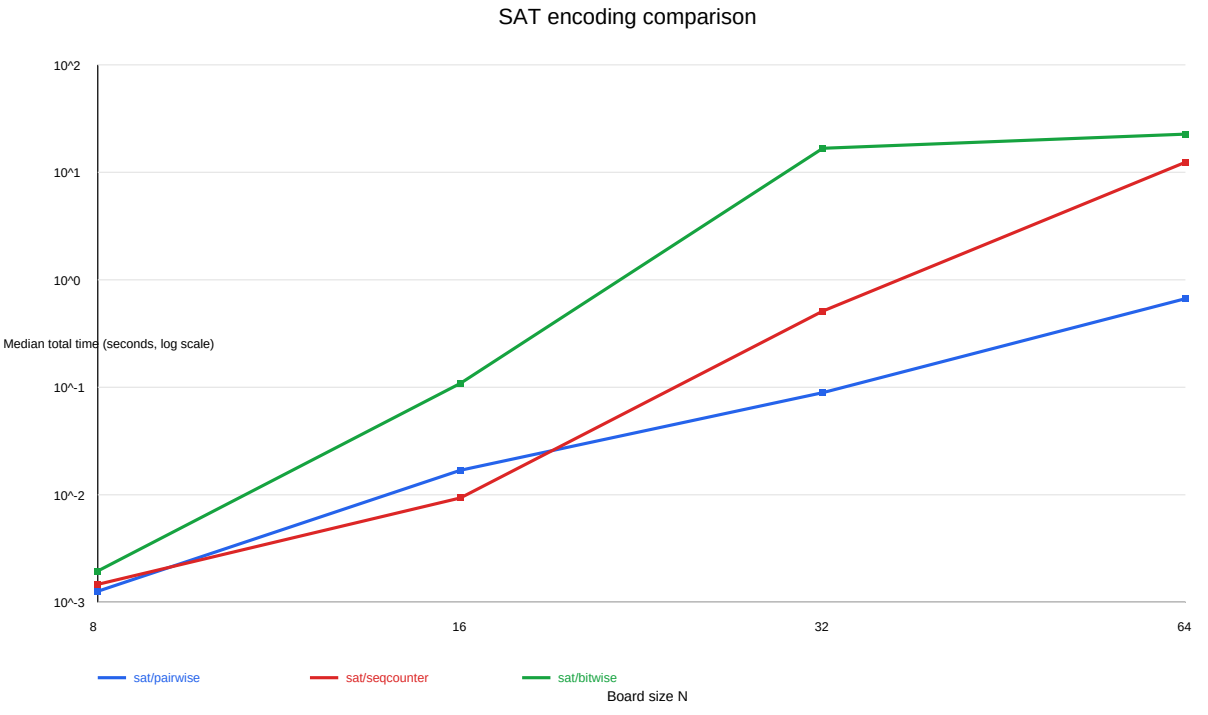


Figure 1: SAT encoding runtime comparison from the repeated local batch.

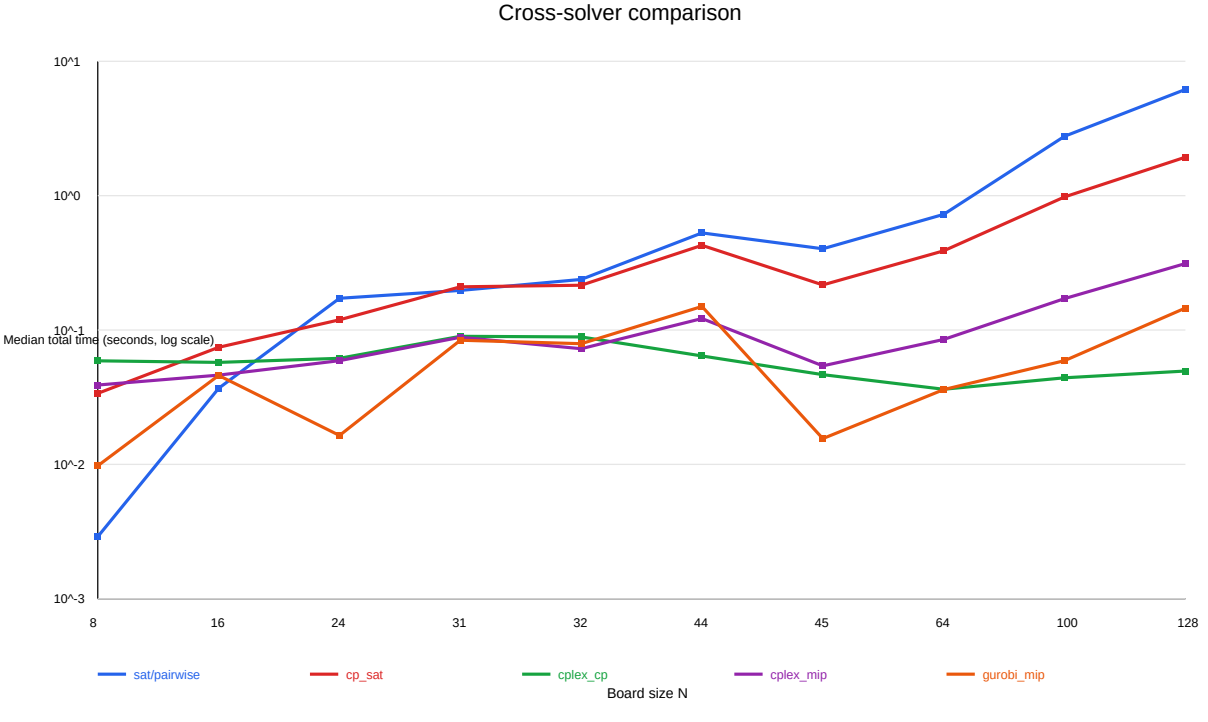


Figure 2: Cross-solver runtime comparison on a logarithmic scale.

7. Limitations and Discussion

The current experiments are limited by the small sample size. CPLEX MIP and CP Optimizer passed 1,100-variable license checks and solved every tested board size through $N = 128$. After academic license activation, Gurobi passed a 2,100-variable license check and solved the previously limited $N = 45$ and larger configurations.

The three-run cross-solver data provide a comparison on one laptop, but it remains a small local sample. The experiment does not control for warm-up, thermal throttling, background load, or variation across machines. All five methods were rerun in the same batch at every tested size from $N = 8$ through $N = 128$.

8. Conclusion

This paper presented a unified implementation for solving N-Queens with SAT, constraint programming, and mixed integer programming. The SAT implementation supports three AMO encodings and includes independent solution validation. The benchmark infrastructure records common performance and status fields and continues after solver-specific errors.

The repeated encoding batch supports Pairwise as a provisional SAT baseline for this Glucose3 implementation: it has the lowest median at three of the four repeatedly measured sizes, while Sequential Counter is faster at $N = 16$. In the matched cross-solver batch, all five methods solve all ten tested sizes through $N = 128$. SAT records the lowest medians at $N = 8, 16$, Gurobi at $N = 24, 31, 45, 64$, CPLEX MIP at $N = 32$, and CPLEX CP at $N = 44, 100, 128$. The three-run, single-machine results support a local comparison rather than a general solver ranking.

Declaration of Competing Interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Data Availability

Source code, benchmark result files, summary tables, and figures are available at: https://github.com/nNm205/n-queens-solving

Declaration of Generative AI and AI-Assisted Technologies

During the preparation of this work, the author used ChatGPT by OpenAI to assist with language revision, code organization, and analysis drafting. The author reviewed and edited the generated material and takes full responsibility for the final content.

References

[1] Gurobi Optimization, LLC, 2026. Gurobi Optimizer Reference Manual, version 13.0. https://docs.gurobi.com.

[2] IBM Corporation, 2026. IBM ILOG CPLEX Optimization Studio documentation, version 22.2. https://www.ibm.com/docs/en/icos.

[3] Ignatiev, A., Morgado, A., Marques-Silva, J., 2018. PySAT: A Python toolkit for prototyping with SAT oracles, in: Theory and Applications of Satisfiability Testing – SAT 2018, Springer. pp. 428–437.

[4] Perron, L., Didier, F., 2026. CP-SAT v9.15. https://developers.google.com/optimization/cp/cp_solver.

[5] Sinz, C., 2005. Towards an optimal CNF encoding of boolean cardinality constraints, in: Principles and Practice of Constraint Programming – CP 2005, Springer. pp. 827–831. doi:10.1007/11564751_73.